

Backend Integration Checklist

Step 1: Firebase Configuration

File: `src/config/firebase.js`

```
1  // Replace MOCK credentials with real ones
2  const firebaseConfig = {
3    apiKey: "YOUR_REAL_API_KEY",           // ← Change this
4    authDomain: "YOUR_PROJECT.firebaseio.com", // ← Change this
5    projectId: "YOUR_PROJECT_ID",         // ← Change this
6    storageBucket: "YOUR_PROJECT.appspot.com", // ← Change this
7    messagingSenderId: "YOUR_SENDER_ID",   // ← Change this
8    appId: "YOUR_APP_ID"                  // ← Change this
9  };
```

How to get credentials:

1. Go to Firebase Console
2. Project Settings > General
3. Scroll to "Your apps" > Web app
4. Copy the config object

Step 2: Backend URL Configuration

File: `src/config/constants.js`

```
1  export const API_BASE_URL = "http://localhost:8080/api"; // ← Verify this
    matches backend
```

Check with backend team:

- What port is Spring Boot running on? (default: 8080)
- What is the API base path? (example: `/api`)

Step 3: Enable Real API Calls

Auth Service - `src/services/authService.js`

Login Function (Line ~50)

```
1  // REMOVE THIS:
```

```
2  const response = await mockApiCall(AUTH_ENDPOINTS.LOGIN, { email, password
  });
3
4  // UNCOMMENT THIS:
5  const response = await axios.post(AUTH_ENDPOINTS.LOGIN, { email, password
  });
6  return response.data;
```

Register Function (Line ~90)

```
1  // REMOVE THIS:
2  const response = await mockApiCall(AUTH_ENDPOINTS.REGISTER, { ... });
3
4  // UNCOMMENT THIS:
5  const response = await axios.post(AUTH_ENDPOINTS.REGISTER, {
6    email,
7    password,
8    firebaseUid
9  });
10 return response.data;
```

User Service - `src/services/userService.js`

Get Blocked Users (Line ~20)

```
1  // REMOVE mock code
2  // UNCOMMENT:
3  const response = await axios.get(USER_ENDPOINTS.GET_BLOCKED);
4  return response.data;
```

Unblock User (Line ~35)

```
1  // UNCOMMENT:
2  const response = await axios.post(`${USER_ENDPOINTS.UNBLOCK}/${userId}`);
3  return response.data;
```

Sync Users (Line ~60)

```
1  // UNCOMMENT:
2  const response = await axios.post(USER_ENDPOINTS.SYNC);
3  return response.data;
```

Signalement Service - `src/services/signalementService.js`

Get All Signalements (Line ~35)

```
1 // UNCOMMENT:
2 const response = await axios.get(SIGNALEMENT_ENDPOINTS.LIST, { params:
  filters });
3 return response.data;
```

Get Stats (Line ~55)

```
1 // UNCOMMENT:
2 const response = await axios.get(`${SIGNALEMENT_ENDPOINTS.LIST}/stats`);
3 return response.data;
```

Update Signalement (Line ~90)

```
1 // UNCOMMENT:
2 const response = await axios.put(`${SIGNALEMENT_ENDPOINTS.UPDATE}/${id}`,
  data);
3 return response.data;
```

Delete Signalement (Line ~105)

```
1 // UNCOMMENT:
2 const response = await
  axios.delete(`${SIGNALEMENT_ENDPOINTS.DELETE}/${id}`);
3 return response.data;
```

Sync Signalements (Line ~120)

```
1 // UNCOMMENT:
2 const response = await axios.post(SIGNALEMENT_ENDPOINTS.SYNC);
3 return response.data;
```

Step 4: Verify Backend API Response Format

Expected Response Structures:

Login/Register

```
1 {
2   "success": true,
3   "user": {
4     "email": "user@test.com",
5     "role": "MANAGER",
6     "firebaseUid": "abc123"
7   },
8   "token": "jwt_token_here" // Optional: if using JWT
```

```
9 }
```

Get Blocked Users

```
1  [  
2    {  
3      "id": 1,  
4      "email": "blocked@test.com",  
5      "role": "USER",  
6      "blocked": true,  
7      "attempts": 3  
8    }  
9  ]
```

Get Signalements

```
1  [  
2    {  
3      "id": 1,  
4      "latitude": -18.8792,  
5      "longitude": 47.5079,  
6      "status": "nouveau",  
7      "date": "2026-01-15",  
8      "surface": 25,  
9      "budget": 500000,  
10     "entreprise": "BuildCo",  
11     "description": "Nid de poule"  
12   }  
13 ]
```

Get Stats

```
1  {  
2    "total": 10,  
3    "totalSurface": 250,  
4    "totalBudget": 5000000,  
5    "progress": 60  
6  }
```

Important: If backend returns different format, update the code that processes responses.

Step 5: JWT Token (If Backend Requires Auth)

Uncomment Axios Interceptor in `src/services/authService.js`

```
1  axios.interceptors.request.use(  
    ,
```

```

2    (config) => {
3        const token = localStorage.getItem('token'); // Saved during login
4        if (token) {
5            config.headers.Authorization = `Bearer ${token}`;
6        }
7        return config;
8    },
9    (error) => {
10       return Promise.reject(error);
11    }
12 );

```

Update AuthContext to Save Token

In `src/contexts/AuthContext.jsx`, login function:

```

1  const login = async (email, password) => {
2      const response = await loginService(email, password);
3      const userData = response.user;
4
5      setUser(userData);
6      localStorage.setItem('user', JSON.stringify(userData));
7
8      // ADD THIS if backend returns token:
9      if (response.token) {
10         localStorage.setItem('token', response.token);
11     }
12
13     return { success: true };
14 };

```

Step 6: Map Offline Tiles (When Ready)

File: `src/pages/MapPage.jsx`

Replace the TileLayer URL:

```

1  <TileLayer
2      // REMOVE online tiles:
3      // url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
4
5      // ADD offline server (ask backend team for URL):
6      url="http://localhost:8081/tiles/{z}/{x}/{y}.png"
7      attribution="Offline Map"
8  />

```

Step 7: Testing with Backend

Test Sequence:

1. Start Backend

```
1  cd ../backend
2  ./mvnw spring-boot:run
```

2. Start Frontend

```
1  npm start
```

3. Test Features:

- ☐ Register new user (check Postgres + Firebase)
- ☐ Login with registered user
- ☐ View map with signalements
- ☐ Manager: Block/unblock user
- ☐ Manager: Edit signalement details
- ☐ Manager: Sync button (check logs)

Step 8: CORS Issues (If Any)

If you see "CORS policy" errors in browser console:

Ask Backend Team to Add CORS Configuration:

```
1  @Configuration
2  public class CorsConfig {
3      @Bean
4      public WebMvcConfigurer corsConfigurer() {
5          return new WebMvcConfigurer() {
6              @Override
7              public void addCorsMappings(CorsRegistry registry) {
8                  registry.addMapping("/api/**")
9                      .allowedOrigins("http://localhost:3000") // React dev
10                     .allowedMethods("GET", "POST", "PUT", "DELETE")
11                     .allowCredentials(true);
12              }
13          };
14      }
15  }
```

Step 9: Common Integration Errors

Error: "Network Error"

Cause: Backend not running or wrong URL

Fix: Check `API_BASE_URL` in constants.js

Error: "403 Forbidden"

Cause: CORS or missing auth token

Fix: Add CORS config + check JWT interceptor

Error: "500 Internal Server Error"

Cause: Backend code issue

Fix: Check backend logs

Error: Data not displaying

Cause: Response format mismatch

Fix: Compare backend response with expected format (Step 4)

Step 10: Final Cleanup

Before deployment:

- ☐ Remove all `console.log('[MOCK API]')` statements
 - ☐ Delete all mock data arrays (mockUsers, mockSignalements)
 - ☐ Test all features with real backend
 - ☐ Update README with final backend URLs
 - ☐ Commit changes to Git
-

Quick Reference

Frontend runs on:

- Development: `http://localhost:3000`
- Docker: `http://localhost:80`

Backend should run on:

- Spring Boot: `http://localhost:8080`
- PostgreSQL: `http://localhost:5432`

Test Login (after backend integration):

- 1 Email: manager@test.com (created by backend default)
- 2 Password: [ask backend team]

Contact Backend Team

Questions to ask:

1. What is the exact backend URL? (`http://localhost:8080/api` ?)
2. Do endpoints return `{ data: [...] }` or just `[...]` ?
3. Is JWT token required? If yes, what header format?
4. What are the default manager credentials?
5. What's the offline tile server URL?